

Relatório Semanal 2 - Projeto Final PHP

Professor: Prof. Alexsander Barreto

Instituição: FAC SENAC DF

Data Limite da Semana: 11/11/2025

Projeto: Kanban Simples Colaborativo

1. Criação da Tela de Login e Validação Front-end

Nesta semana, o foco foi na implementação da interface de acesso e na garantia da integridade inicial dos dados através da validação no lado do cliente. A tela de login foi criada no arquivo `auth/login.php`.

1.1. Estrutura da Tela de Login (`auth/login.php`)

A tela de login é composta por um formulário HTML simples, estilizado para ser responsivo. O formulário utiliza o método `POST` para enviar as credenciais para o próprio script PHP para processamento.

Código Comentado (Estrutura HTML/PHP)

```
<?php
// auth/login.php

// 1. Inclusão de arquivos de configuração e utilidades
require_once '../config/config.php'; // Configurações gerais
require_once '../includes/functions.php'; // Funções de utilidade (e.g.,
redirecionamento)

// 2. Lógica de processamento do formulário
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $email_usuario = $_POST['email_usuario'] ?? '';
    $senha = $_POST['senha'] ?? '';

    // Validação básica de preenchimento (o PHP fará a validação de
    segurança)
    if (empty($email_usuario) || empty($senha)) {
        $erro = "Por favor, preencha todos os campos.";
    } else {
        // Tenta autenticar o usuário (lógica detalhada na próxima seção)
        // ...
    }
}
?>
<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="UTF-8">
    <title>Login - Kanban Simples</title>
    <!-- Inclusão de CSS (assets/css/style.css) -->
</head>
<body>
    <div class="login-container">
        <h1>Acesso ao Kanban Simples</h1>
        <?php if (isset($erro)): ?>
            <p class="error-message"><?= $erro ?></p>
        <?php endif; ?>

        <!-- O formulário aponta para o próprio arquivo para processamento -
        ->

        <form action="login.php" method="POST">
            <label for="email_usuario">E-mail ou Usuário:</label>
            <!-- O atributo 'required' habilita a validação front-end do
            HTML5 -->
```

```
<input type="text" id="email_usuario" name="email_usuario"
required>

<label for="senha">Senha:</label>
<input type="password" id="senha" name="senha" required>

<button type="submit">Entrar</button>
</form>

<p>Não tem conta? <a href="register.php">Registre-se aqui</a>.</p>
</div>
</body>
</html>
```

1.2. Validação Front-end

A validação inicial no lado do cliente foi implementada utilizando os recursos nativos do **HTML5**, especificamente o atributo `required` nos campos de entrada. Isso garante que o navegador impeça o envio do formulário caso os campos estejam vazios, proporcionando um *feedback* imediato ao usuário e reduzindo requisições desnecessárias ao servidor.

2. Conexão Inicial com Banco de Dados (MySQL)

A conexão com o banco de dados MySQL é fundamental para a autenticação. O arquivo `config/db.php` foi criado para centralizar as credenciais e estabelecer a conexão.

2.1. Explicação da Conexão MySQL (`config/db.php`)

O script utiliza a extensão **MySQLi** (MySQL Improved) com o paradigma Orientado a Objetos para criar uma conexão persistente com o banco de dados.

Código Comentado (Conexão MySQL)

```
<?php
// config/db.php

// 1. Definição das constantes de conexão
define('DB_HOST', 'localhost');
define('DB_USER', 'kanban_user'); // Usuário com permissões mínimas
define('DB_PASS', 'senha_segura'); // Senha do usuário
define('DB_NAME', 'kanban_db'); // Nome do banco de dados

// 2. Tentativa de conexão
$conn = new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME);

// 3. Verificação de erro na conexão
if ($conn->connect_error) {
    // Em ambiente de produção, esta mensagem deve ser genérica
    // e o erro deve ser logado.
    die("Falha na conexão com o banco de dados: " . $conn->connect_error);
}

// 4. Configuração do charset para UTF-8
$conn->set_charset("utf8");

// A variável $conn agora contém o objeto de conexão e será incluída
// em outros scripts que precisem interagir com o banco de dados.
?>
```

2.2. Testes Funcionais Básicos (Simulação de Autenticação)

Para testar a funcionalidade da conexão, um teste básico de autenticação foi simulado no `auth/login.php`.

Teste: Verificar se um usuário existe no banco de dados e se a senha fornecida corresponde ao *hash* armazenado.

```
// Trecho de código simulado em auth/login.php, após a validação de campos

// 1. Incluir a conexão com o banco de dados
require_once '../config/db.php';

// 2. Preparar a consulta para evitar injeção de SQL
$stmt = $conn->prepare("SELECT id, senha_hash FROM usuarios WHERE email = ?
OR nome = ?");
$stmt->bind_param("ss", $email_usuario, $email_usuario);
$stmt->execute();
$result = $stmt->get_result();

if ($result->num_rows === 1) {
    $usuario = $result->fetch_assoc();

    // 3. Verificar a senha usando password_verify()
    if (password_verify($senha, $usuario['senha_hash'])) {
        // Autenticação bem-sucedida: Iniciar sessão e redirecionar
        session_start();
        $_SESSION['user_id'] = $usuario['id'];
        header("Location: ../dashboard/index.php");
        exit();
    } else {
        $erro = "Credenciais inválidas.";
    }
} else {
    $erro = "Credenciais inválidas.";
}
$stmt->close();
```

3. Capturas de Tela (Simuladas)

Descrição	Imagem (Representação Textual)
Tela de Login	[Imagem 1: Tela de Login com campos "E-mail ou Usuário" e "Senha", e botão "Entrar". Abaixo, link "Registre-se aqui".]
Mensagem de Erro	[Imagem 2: Tela de Login exibindo a mensagem de erro: "Credenciais inválidas." em destaque.]
Conexão de Sucesso	[Imagem 3: Tela do console do navegador ou terminal mostrando a mensagem de sucesso da conexão com o BD (após um teste de conexão direto).]

4. Dificuldades Encontradas

Durante a implementação desta fase, duas dificuldades principais foram identificadas e resolvidas:

Dificuldade	Descrição e Solução
Segurança da Senha	Descrição: A tentativa inicial é armazenar senhas em texto puro. Solução: Foi implementado o uso obrigatório da função nativa do PHP <code>password_hash()</code> para armazenar senhas de forma segura no banco de dados e <code>password_verify()</code> para a checagem durante o login.
Injeção de SQL	Descrição: O uso direto de variáveis de formulário em consultas SQL. Solução: A conexão MySQLi foi utilizada com Prepared Statements (<code>\$conn->prepare()</code>), garantindo que todos os dados de entrada do usuário sejam tratados como <i>strings</i> e não como comandos SQL, prevenindo ataques de injeção.
Redirecionamento de Sessão	Descrição: Garantir que o usuário seja redirecionado corretamente para o login ao acessar a raiz (<code>index.php</code>) e para o <i>dashboard</i> após o sucesso. Solução: O arquivo <code>index.php</code> foi configurado para verificar a sessão e usar a função <code>header("Location: ...")</code> para o redirecionamento, com <code>exit()</code> logo em seguida para garantir que nenhum código posterior seja executado.

Referências

- [1] [GitHub - lucasparente-codigos/kanban-simples-php](#) - Repositório do projeto “Kanban Simples Colaborativo” .